IoT & Cloud for sustainable communities

Exercise lessons

Introduction

I'm Pietro Rando Mazzarino, post-doc Researcher at the Energy Center Facility of Politecnico di Torino.

email : pietro.randomazzarino@polito.it

Classes : 30 hrs in presence

Tutoring session : 30 hrs

Materials :

- slides will be uploaded on the Portale della didattica
- any communication will be done through the notice section of the course page in Portale della didattica
- Codes/class exercises etc (anything code related) you can find in this github repo

Rules & Tips

1. **ATTEND CLASSES!** joining regularly exercises classes will make it easier for all.
2. **CODE!** you have to code and exercise to succeed. always bring you PC to classes and stay up to date with the exercises.
3. **TUTORING SESSIONS.** The exact starting period for tutoring slots will be communicated and must be respected. I'll fix tutoring slots and i strongly recommend you and your groups to use them because **no additional time will be granted,** for corrections or counseling. Tutoring session must be reserved in advance via email at least one day before, try to organize yourselves and come in groups. Sessions that are not reserved by anyone will not be granted.
4. **Google and LLM Agents**: they are the programmers best friend if used correctly… do not simply copy and paste but UNDERSTAND!
5. **COMMUNICATIONS**: individual communications to me must have Professor Bottaccioli in cc. always use "reply to all" when replying to emails! Group communications must have all the member in cc and a clear statement of the group number in the email object

Tutoring sessions 2026/2027



The tutoring sessions are slots in which you could come to solve any of your doubts on programming, project etc.. TRY TO JOIN IN GROUPS WITH CLEAR DOUBTS AND QUESTIONS!

THE PRESENCE AT A TUTORING SESSION MUST BE NOTIFIED VIA EMAIL TO ME AT LEAST ONE DAY IN ADVANCE —> NO EMAIL NO TUTORING SESSION, LATE EMAIL NO TUTORING SESSION!



Tutoring sessions 2026/2027

Tutoring slots are from 16:00 to 19:00 in the following dates :

- 3 Dec
- 10 Dec
- 17 Dec
- 7 Jan
- 14 Jan
- 21 Jan
- 28 Jan
- 4 Feb
- 11 Feb
- 18 Feb

where? :

Energy center, 2nd floor, via paolo Borsellino 38/16

Exam structure

1. Written exam 40% : 6 multiple choice questions + 2 open theory questions + 1 simple coding exercise on REST or MQTT

2. Group Project 60% : must be a functioning Iot project example + a presentation of the project. Be creative and try to enjoy

They can be taken in different exam sessions and we keep valid the grade of each one for a maximum of one year.

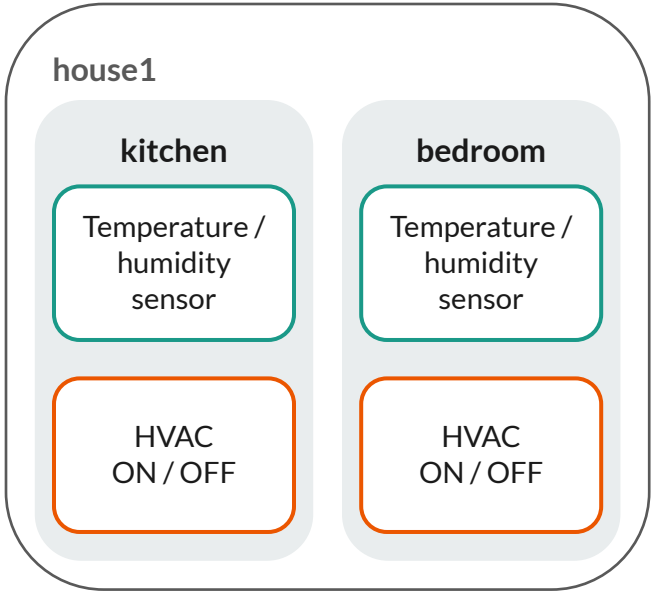
Course roadmap: one project, 13 lessons

1. L01 Intro, environment, Python refresh
2. L02-L03 OOP: devices, device connector, controller
3. L04-L06 REST: CherryPy, device REST API, Catalog
4. L07-L08 MQTT: paho, MyMQTT, SenML, controller
5. L09 ThingSpeak adaptor
6. L10-L11 Telegram bot
7. L12 Integration, L13 Revision

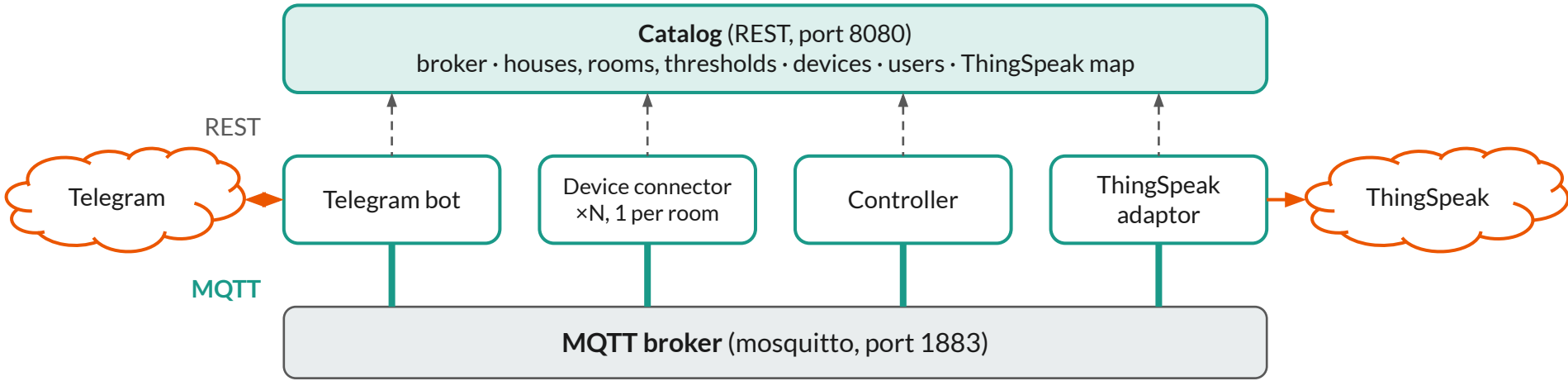
Every lesson adds one module to the same project. ALL of this is what you need for the exam project!

The course project: a smart home

- Each room has a temperature/humidity sensor and an ON/OFF HVAC (heater).
- Goal: control the temperature, warn on high humidity.
- Must scale: many houses, rooms added at runtime, no code changes.
- Small independent microservices talking only through REST and MQTT.



Architecture of the project



REST = who / where / settings. MQTT = data flow.

Getting Started - Python development

Setting the environment to start coding

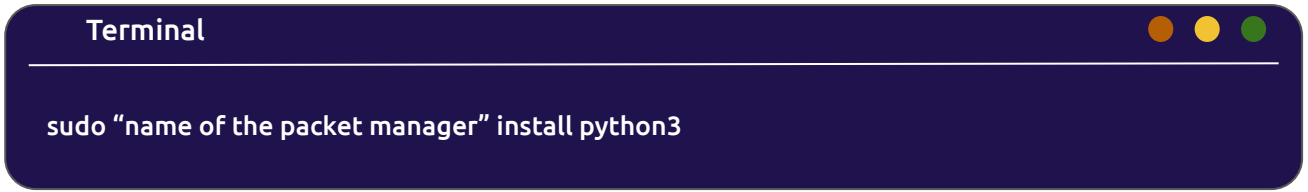
Getting started

- Install Python 3.12 (the course version) from python.org for your operating system.
- If not in Linux, directly launch the installer (next slide for linux installation)
- Pay attention to select “add Python to path”
- Wait until finished

Installing on Linux

Python3 should be already available on most Linux distributions.

In case you don’t have it you can install it by writing on a terminal:



- For example in Ubuntu —> ***sudo apt-get install python3***

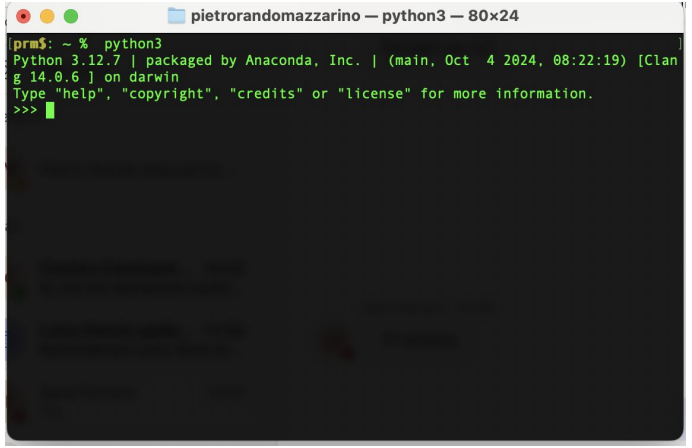
Check installation

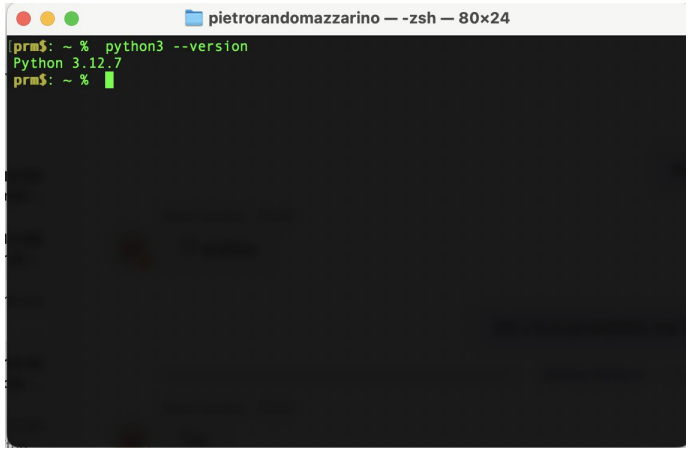
Open the terminal and try one of the following commands:

- python3
- python3 --version (must say 3.12.x; with several versions: python3.12 --version)

The first one will open the interactive shell of python interpreter, the second one will simply return the version of installed python interpreter.

NB to close the interactive python shell type: *quit()* and press enter





IDE - Integrated Development Environment

Suggested : VS code —> The classes will be done using VS code so please try to be compliant

others: Pycharm, Spider etc..

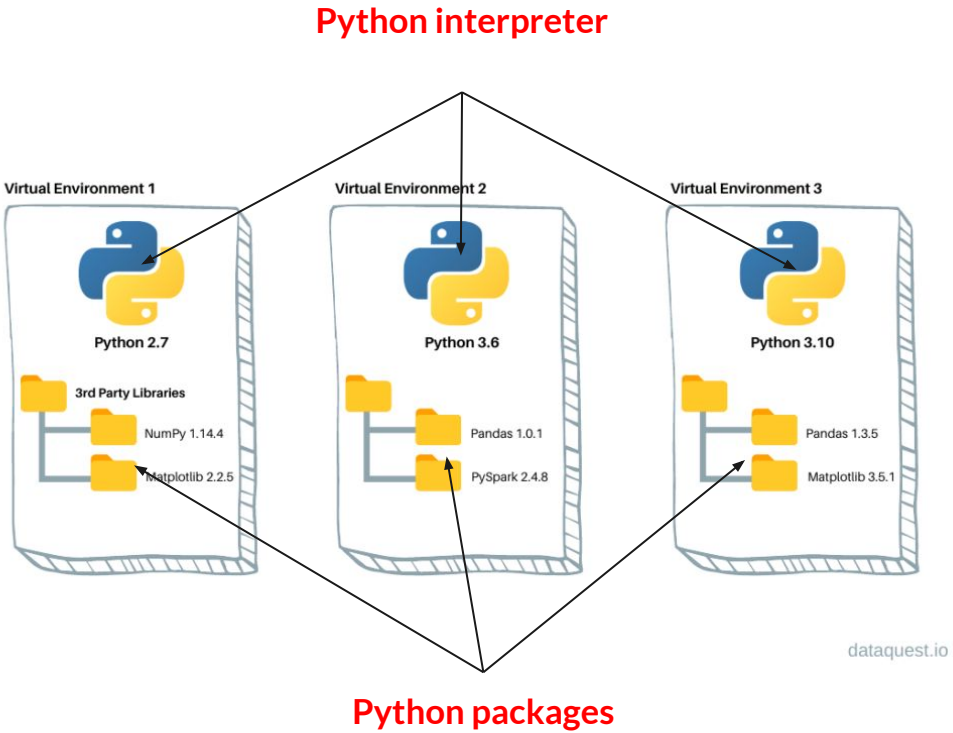
Vibe coding only if you can already understand codes otherwise is useless!!

For VS code you should have everything ready (from the prior Python course) if not, do that course before!!

Environments

- **What is a Python Environment?**
  - An isolated workspace to install packages without affecting other projects.
  - Ensures dependency management and version control.

- **Types of Environments**
  - Global environment: shared by all projects **(NEVER USE THIS)**
  - Virtual environment (`venv`): project-specific and lightweight.
    - doc: HERE
  - Conda environment: used with Anaconda/Miniconda.
    - doc: HERE



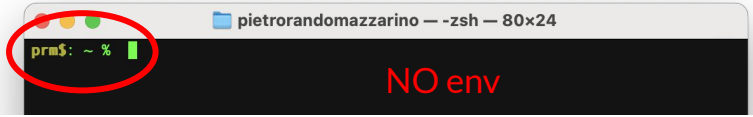
Environments

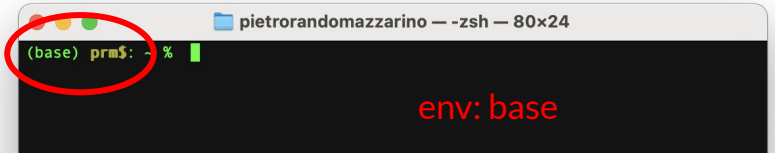
Why we use environments?

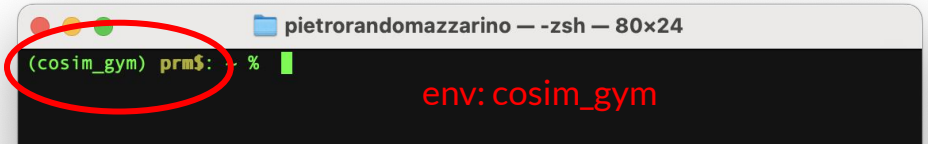
On of the most powerful features of Python programming, besides its simple syntax, relies on the existence of thousand of libraries/packages that could be installed and imported. Depending on the project we could use different version of the same library or libraries that are conflictual. To ensure that conflicting libraries are separated, to keep track of what dependencies we need for a project and what for another the usage of environments is fundamental!

Tips:

- Always be sure to run scripts and install packages when you are inside the right environment (this means you use a specific interpreter and a specific package manager that only operates inside the env)







VS code select the right environment

VS code as most of the modern IDE allows to directly manage/create and select environment using its interface. This a facilitation approach and i attach the reference on how to here.

Nevertheless remember that creation of an environment can be done externally directly from command line and then it is possible to select the environment interpreter. This will ensure that any run you do from within the project will use the right selected python interpreter!!

In VS Code: Cmd/Ctrl+Shift+P → Python: Select Interpreter → choose .venv. Open a new terminal: the prompt starts with (.venv).

Create the environment with venv

```
cd <project-folder>
python3 -m venv .venv              # Windows: py -m venv .venv
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\Activate.ps1         # Windows PowerShell
pip install -r requirements.txt
python --version                   # check the version
deactivate                         # leave the environment
```

requirements.txt = the list of libraries the project needs, one per line. Create it from an active environment with `pip freeze > requirements.txt`.

Create the environment with conda

```
conda create -n <env-name> python=<version>
conda activate <env-name>          # Windows: Anaconda Prompt
cd <project-folder>
pip install -r requirements.txt    # or: conda install <package>
python --version                   # check the version
conda env list                     # list all environments
conda deactivate                   # leave the environment
```

environment.yml = conda's equivalent of requirements.txt. Export it with `conda env export > environment.yml`, recreate it with `conda env create -f environment.yml`.

Exercise 1 - functions

File: L01_intro/solution/ex1_functions.py (you work in starter/)

- celsius_to_fahrenheit(c)
- is_too_cold(temp, t_min) and is_too_hot(temp, t_max)
- hvac_command(temp, t_min, t_max) → "ON" if too cold, "OFF" if too hot, None inside the band

This is the rule of the project's controller.

Exercise 2 - read the configuration

File: L01_intro/solution/ex2_config.py, with config.json

- load_config(path): JSON file → dict
- list_rooms(config): "house1/kitchen", ...
- get_thresholds(config, house_id, room_id), or None

```
with open(path) as f:
    return json.load(f)
```

Tip: Path(__file__).parent makes paths work from any folder.

Exercise 3 - modify and save

File: L01_intro/solution/ex3_save.py

- Change a threshold in the loaded dict.
- Write the result to a NEW file, config_modified.json.

Never overwrite config.json: L03 loads the rooms from it.

```
with open(path, "w") as f:
    json.dump(config, f, indent=2)
```